

LabRobFail: A Benchmark for Robotic Failure Analysis in Chemical Self-driving Laboratories

Haobo Wang^{1,2*}, Baoli Sun^{1*}, Anqi Zou^{1,2}, Dongsheng Huang¹, Zelin Lv¹, Ning Wang¹,
Rui Li^{3,4}, Dongzhan Zhou^{4†}, Weiye Guo^{5†}, Zhihui Wang^{1,2†}, Wanli Ouyang^{2,4,6†}

¹School of Software, Dalian University of Technology, Dalian, China

²Shenzhen Loop Area Institute-Hong Kong Science and Technology Innovation Cooperation Zone, Shenzhen, China

³School of Computer Science and Engineering, Harbin Institute of Technology, Harbin, China

⁴Shanghai AI Laboratory, Shanghai, China

⁵The Hong Kong University of Science and Technology, Hong Kong, China

⁶The Chinese University of Hong Kong, Hong Kong, China

dongzhan.zhou@gmail.com, guoweiyu96@gmail.com, zhwang@dlut.edu.cn, wlouyang@ie.cuhk.edu.hk

Abstract

The deployment of embodied agents in self-driving laboratories could accelerate scientific discovery, yet their reliability is constrained by the irreversible and safety-critical nature of chemical experiments. Progress is further hindered by scarce failure data and the lack of fine-grained evaluation protocols. To address these challenges, we introduce LabRobFail, a failure-centric framework for learning and evaluating robotic failure analysis in chemical laboratories. LabRobFail-Sim injects controllable failures at the control, physics, and semantic levels, enabling the construction of LabRobFail-Data, which contains over 20,000 trajectories across 70+ task scenarios, five failure categories, and 11 fine-grained failure types. LabRobFail-Bench evaluates six capabilities spanning task understanding, failure detection, temporal localization, severity assessment, failure classification, and actionable correction. We further develop LabRobFail-VLM, a domain-specialized vision-language model that generates structured failure diagnoses and recovery instructions. On seen environments, it achieves 92.58% failure-detection accuracy and 85.58% temporal-localization accuracy, substantially outperforming general-purpose VLMs. When integrated as a real-time supervisor, it improves downstream VLA task success rates by 10–20 percentage points, demonstrating the value of fine-grained failure understanding for closed-loop recovery and reliable laboratory autonomy. Our code and data are available at <https://github.com/Su-ISE-2001/SciRobo>.

Introduction

The emergence of Self-Driving Laboratories (SDLs) (Abolhasani and Kumacheva 2023; Lan et al. 2025; Li et al. 2025; Szymanski et al. 2023) offers a promising route to accelerate scientific discovery through embodied automation. Although recent Vision-Language-Action (VLA) models (Black et al. 2025; Kim et al. 2024; Zhen et al. 2024; Zitkovich et al. 2023) have shown strong generalist manipulation capabilities, their deployment in chemical laboratories remains constrained by reliability. Unlike household tasks, where failures are often reversible (Duan et al. 2024; Liu, Bahety, and

Song 2023; Pacaud et al. 2025), laboratory experiments follow strict protocols and involve irreversible processes and substantial safety risks. Minor deviations, such as pipette misalignment or excessive agitation, may contaminate samples, invalidate long-horizon workflows, or cause hazardous incidents. Therefore, reliable laboratory autonomy requires agents not only to execute tasks, but also to perceive, diagnose, and recover from failures in real time (Kambhampati et al. 2024; Lin et al. 2023; Qu et al. 2024).

While recent studies have applied Vision-Language Models (VLMs) to robotic failure detection (Duan et al. 2024; Guo et al. 2024; Zhou et al. 2025), their extension to self-driving laboratories is limited by two key gaps. **(1) The scarcity of large-scale failure data.** Real-world failure collection is costly and hazardous, while existing scientific simulators such as LabUtopia (Li et al. 2025) and AutoBio (Lan et al. 2025) primarily emphasize successful execution and lack systematic failure-injection pipelines. Consequently, current agents are trained largely on successful trajectories, leaving insufficient coverage of diverse laboratory anomalies. **(2) The coarseness of existing evaluation benchmarks.** Existing benchmarks, largely designed for household manipulation (Duan et al. 2024; Mitash et al. 2023; Thoduka et al. 2024), mainly assess binary detection or coarse failure categories, overlooking fine-grained localization, classification, and severity assessment. Although some methods provide corrective feedback (Duan et al. 2024; Lu et al. 2025), their outputs are often ambiguous and difficult to convert into executable recovery actions. This limitation is especially critical in chemical experiments, where irreversible processes demand precise, risk-aware intervention rather than trial-and-error.

To address these gaps, we introduce LabRobFail, a failure-centric framework for robotic failure analysis in self-driving laboratories, comprising a simulation platform, a large-scale dataset, and a multi-dimensional benchmark. First, building upon the physical realism established by LabUtopia, we develop **LabRobFail-Sim**, which extends these environ-

*These authors contributed equally.

†Corresponding author.

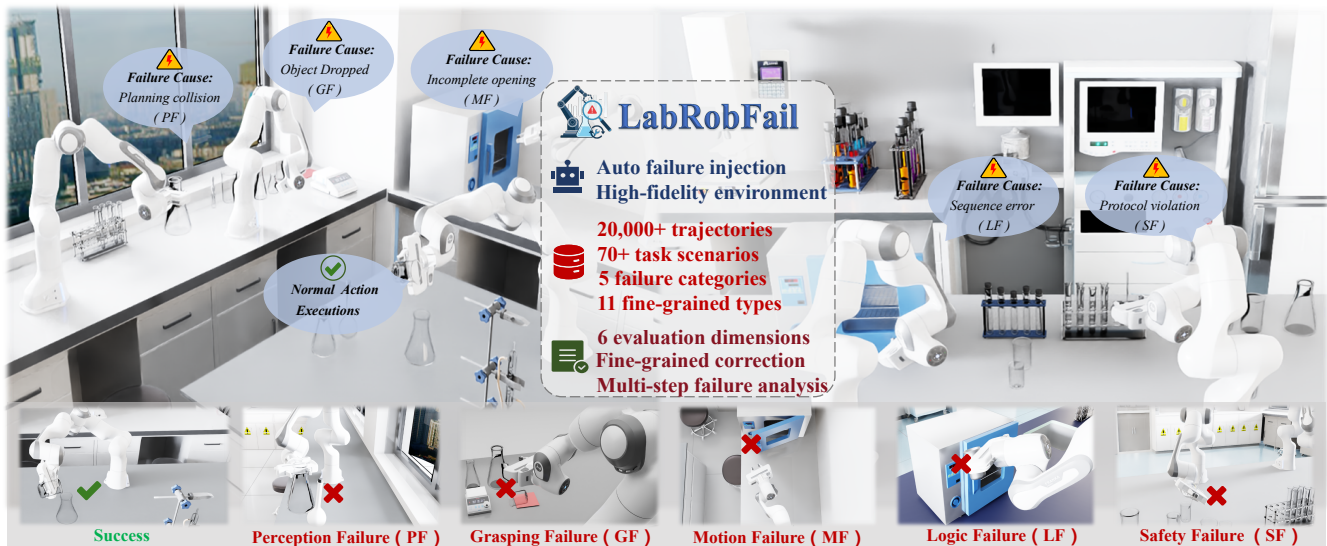


Figure 1: **LabRobFail Framework for Chemical Laboratory Failure Analysis.** A high-fidelity simulation platform automatically injects failures across five categories: Perception (PF), Grasping (GF), Motion (MF), Logic (LF), and Safety (SF). The resulting data support a six-dimensional benchmark for failure detection, localization, diagnosis, and fine-grained correction.

ments with a novel automated failure injection pipeline. Instead of relying on manually designed faults, LabRobFail-Sim systematically perturbs control signals, physical dynamics, and task-level conditions to generate large-scale annotated failure trajectories covering execution, perception, and semantic anomalies. Based on this pipeline, we construct **LabRobFail-Data**, comprising 5 major failure categories (Perception, Grasping, Motion, Logic, and Safety), 11 fine-grained failure types, and 70+ task instances ranging from short-horizon manipulations to long-horizon multi-stage workflows, totaling over 20,000 operation trajectories. Crucially, the dataset balances successful executions with diverse failure modes, enabling models to learn the precise discriminative boundary between normal operations and subtle anomalies. Finally, we design **LabRobFail-Bench**, the first multi-dimensional failure understanding benchmark tailored for chemical laboratories. LabRobFail-Bench provides systematic evaluation covering failure detection, localization, mode classification, severity assessment, and fine-grained correction planning. It directly addresses the stringent safety requirements of laboratory automation, paving the way for closed-loop recovery in high-stakes scientific experiments.

Building on LabRobFail-Data, we introduce LabRobFail-VLM, which formulates laboratory failure analysis as structured prediction over failure presence, temporal localization, type, severity, and corrective guidance. Beyond single-label detection, it produces semantically grounded, recovery-oriented diagnoses and serves as a task-specific supervisor linking failure understanding to actionable recovery. Experiments show that LabRobFail-VLM consistently outperforms general-purpose VLMs and improves downstream policy recovery when integrated into the control loop.

In summary, (1) We introduce LabRobFail-Sim, a failure-centric simulation framework that enables scalable and controllable failure generation through automated perturbations

at the control, physics, and semantic levels. (2) We construct LabRobFail-Data and LabRobFail-Bench, establishing the first large-scale dataset and multi-dimensional evaluation standard for failure reasoning in chemical laboratory environments. (3) We develop LabRobFail-VLM, demonstrating that domain-specialized failure analysis can support actionable diagnosis and closed-loop recovery.

Related Work

Embodied AI and Simulation for SDLs

Scientific automation is evolving from rigid, script-based systems toward embodied agents. While recent works like Chemputer (Steiner et al. 2019) and Artificial Chemist (Epps et al. 2020) standardized synthesis via closed-loop optimization, training generalist agents requires high-fidelity simulators beyond physical hardware. Recent platforms such as AutoBio (Lan et al. 2025) and LabUtopia (Li et al. 2025) have advanced this frontier by simulating biological micro-manipulation and multi-physics chemical interactions, respectively. However, these environments focus on successful execution and lack systematic failure injection, leading to a survival bias toward ideal trajectories. LabRobFail addresses this gap by introducing a controllable anomaly generation pipeline for safety-critical failures in scientific workflows.

Robotic Failure Understanding and Benchmarks

The integration of VLMs has shifted robotic failure analysis from fixed sensor thresholds toward semantic reasoning. REFLECT (Liu, Bahety, and Song 2023) used LLMs for retrospective log-based diagnosis, while AHA (Duan et al. 2024) introduced FailGen to generate large-scale failure data through procedural perturbations. Later studies improved failure granularity: RoboFAC (Lu et al. 2025) developed a hierarchical taxonomy of planning and execution errors, and

Table 1: Comparison of robotic failure detection datasets and benchmarks. Lab Env. / Lab Inst.: Support for chemical laboratory environments and specialized laboratory instruments. Auto Inj.: Automated failure injection. # Fail. Types / # Eval Dims: Number of failure categories and evaluation dimensions. Fine Corr. / Temp. Loc. / Severity: Fine-grained correction, temporal localization, and severity assessment capabilities. The **best** and second-best results are highlighted.

Dataset/Benchmark	Laboratory Support		Data Statistics				Detection & Correction Capability		
	Lab Env.	Lab Inst.	Auto Inj.	# Failure Types	# Eval Dims	# Traj.	Fine Corr.	Temp. Loc.	Severity
ARMBench Video Defect (Mitash et al. 2023)	✗	✗	✗	2	1	4,070	✗	✗	✗
ViFailback dataset (Zeng et al. 2025)	✗	✗	✗	4	<u>2</u>	5,202	✗	✗	✗
RLBench-Fail (Pacaud et al. 2025)	✗	✗	✓	5	1	14,358	✗	✗	✗
BridgeDataV2-Fail (Pacaud et al. 2025)	✗	✗	✓	5	1	9,830	✗	✗	✗
UR5-Fail (Pacaud et al. 2025)	✗	✗	✓	5	1	570	✗	✗	✗
SMF-DROID (Grislain et al. 2025)	✗	✗	✓	1	<u>2</u>	6,276	✗	✗	✗
RoboFAC (Lu et al. 2025)	✗	✗	✓	6	<u>2</u>	10,722	✗	✗	✗
FAILURE (Thoduka et al. 2024)	✗	✗	✗	5	<u>2</u>	229	✗	✗	✗
AHA (Duan et al. 2024)	✗	✗	✓	<u>7</u>	<u>2</u>	49K	✗	✗	✗
LabRobFail-Data (Ours)	✓	✓	✓	11	6	20K	✓	✓	✓

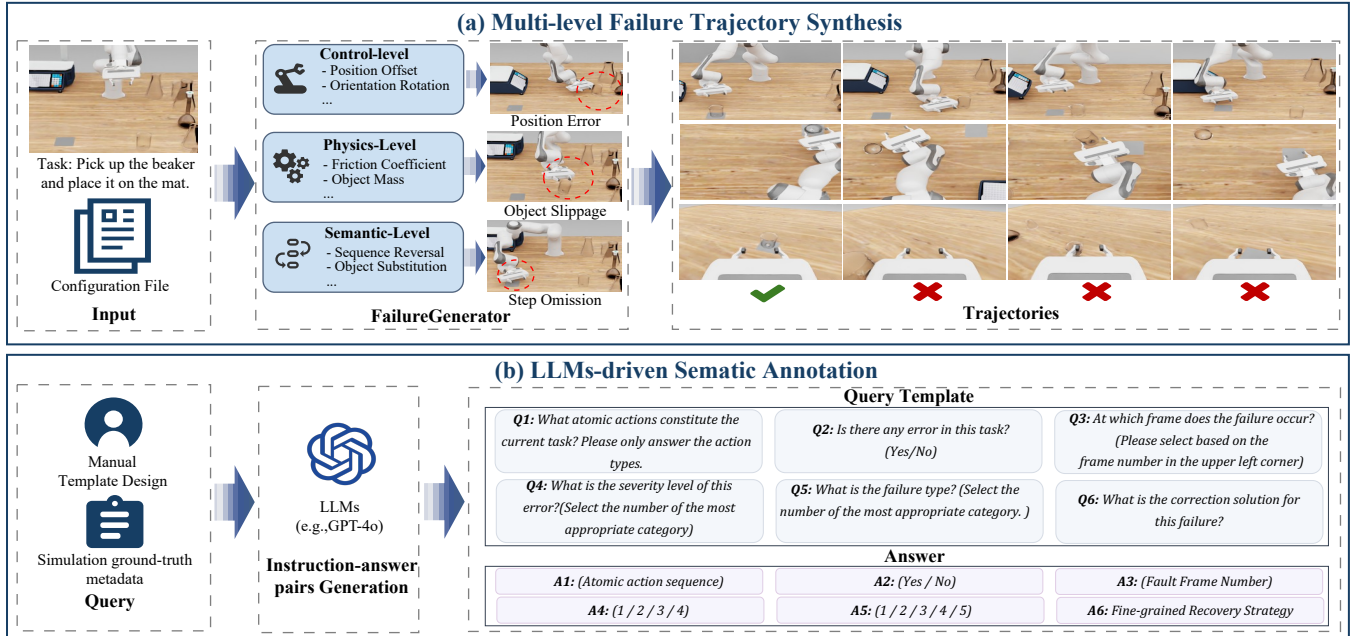


Figure 2: **The LabRobFail-Sim Framework.** (a) The multi-level failure generator injects faults via Control, Physics, and Semantic perturbations. (b) The LLM-driven sematic annotation pipeline leverages simulation metadata and GPT-5.4 to automatically generate instruction-answer pairs.

Guardian (Pacaud et al. 2025) extended failure categorization across multiple manipulation benchmarks (James et al. 2020; Walke et al. 2023). However, existing benchmarks remain centered on household settings with largely reversible failures and provide limited support for precise recovery. LabRobFail-Bench addresses this gap by evaluating failure detection, localization, severity assessment, and fine-grained correction in safety-critical chemical laboratories.

LabRobFail

LabRobFail-Sim

Although LabUtopia (Li et al. 2025) provides high-fidelity simulation of chemical interactions, it primarily focuses on successful task execution. Built on its physics engine, we develop LabRobFail-Sim to model the stochastic failures of real laboratories through an automated pipeline comprising

multi-level failure trajectory synthesis and LLM-driven semantic annotation. Together, these modules enable the scalable generation and annotation of LabRobFail-Data.

Multi-level Failure Trajectory Synthesis In Fig. 2 (a), to facilitate controllable generation, we formulate a robot manipulation skill as a sequence of sparse keyframes $\tau = \{(T_i, g_i, o_i)\}_{i=1}^N$, where $T_i = (R_i, p_i) \in SE(3)$ denotes the end-effector’s target pose (rotation matrix R_i and translation vector p_i), $g_i \in \{0, 1\}$ represents the gripper status, and o_i is the interaction target. The **FailureGenerator** module synthesizes failure trajectories $\hat{\tau}$ by applying systematic perturbation functions Φ across three hierarchical levels.

(1) Control-level Perturbation (Φ_{ctrl}): We simulate execution errors by injecting stochastic noise into keyframe parameters. Specifically, we apply Gaussian perturbation to

the translation p_i and Lie algebra noise to the rotation R_i :

$$\tilde{p}_i = p_i + \xi_{trans}, \quad \tilde{R}_i = R_i \cdot \text{Exp}(\xi_{rot}), \quad (1)$$

where $\xi \sim \mathcal{N}(0, \Sigma)$. Gripper commands are corrupted with a failure probability λ_{grip} to model actuator faults.

(2) Physics-level Perturbation (Φ_{phy}): To induce emergent failures like slippage or grasp instability, we dynamically adjust the simulation dynamics parameters $\Psi = \{\mu, m, \nu, \dots\}$ (representing friction, mass, viscosity, etc.). The perturbed parameters $\tilde{\Psi}$ are derived via uniform scaling:

$$\tilde{\psi} = \psi \cdot (1 + \delta), \quad \text{with } \delta \sim \mathcal{U}(-\alpha, \alpha). \quad (2)$$

This ensures that the environment dynamics deviate from the nominal model, testing the agent’s physical robustness.

(3) Semantic-level Perturbation (Φ_{sem}): This level introduces task logic errors. We model this as a permutation operator π on the sequence indices to simulate step reversals or skips ($\tilde{\tau}_{seq} = \{T_{\pi(1)}, \dots\}$), and a mapping function $\mathcal{M}(o_i)$ for target object substitution (e.g., selecting the wrong reagent), leading to safety constraint violations.

The trajectory $\tilde{\tau} = \Phi_{sem}(\Phi_{ctrl}(\tau))$ is executed under perturbed dynamics $\Phi_{phy}(\Psi)$. Adopting a scalable **configuration-driven design**, we specify perturbable keyframes and ranges α to generate labeled trajectories.

LLM-driven semantic annotation Because manual annotation of temporal and semantic labels is impractical at scale, we develop a GPT-5.4-based VQA annotation pipeline, as shown in Fig. 2(b). Using simulation ground-truth metadata, it instantiates query and correction templates aligned with the six dimensions of LabRobFail-Bench to generate context-specific instruction–answer pairs. The annotations are then validated through rule-based filtering and human sampling before being aligned with failure trajectories to construct LabRobFail-Data. Prompting and verification details are provided in the Supplementary Material.

LabRobFail-Data

Based on LabRobFail-Sim, we construct LabRobFail-Data, a large-scale dataset for laboratory robotic failure analysis in chemical laboratories. It comprises over **20,000** interaction trajectories across **70+** distinct laboratory task scenarios.

Task Complexity Levels. As shown in Fig. 3(a), we organize experiment tasks into three complexity levels: **(1) Short-horizon Tasks (Sh. Task, ≤ 10 frames):** Focus on atomic operations such as *pick*, *place*, and *pour*. Failures here are typically instantaneous execution errors. **(2) Medium-horizon Tasks (Mh. Task, 10-30 frames):** Chain 2-3 atomic operations (e.g., *stir the beaker with glass rod*) to examine failure detection during critical action transitions. **(3) Long-horizon Tasks (Lh. Task, > 50 frames):** Involve complex multi-stage workflows (e.g., *pick beaker in dryer*) that require hierarchical planning and precise fault localization. To facilitate robust learning on complex workflows, we explicitly oversample medium and long-horizon tasks to ensure sufficient coverage for multi-step tasks.

Fine-grained Failure Taxonomy. We design a systematic taxonomy covering **5 major categories** and **11 fine-grained**

types (Fig. 3(b)), spanning the full spectrum of anomalies: **(1) Perception Failure (PF):** Arises from the challenging optical properties of materials. It includes *Target Position Deviation* and failures caused by transparent glassware or reflective fluid surfaces. **(2) Grasping Failure (GF):** Reflects end-effector control deficiencies, such as *Object Slippage* due to varying friction coefficients of wet surfaces or insufficient gripping force. **(3) Motion Failure (MF):** Corresponds to trajectory execution anomalies. A domain-specific highlight is *Incomplete Trajectory*, where premature termination of motion sequences leads to experimental failure. **(4) Logic Failure (LF):** Captures semantic planning errors (e.g., *Step Omission*, *Sequence Reversal*) that invalidate experimental outcomes. **(5) Safety Failure (SF):** This category covers *Improper Handling* and *Protocol Violations* (e.g., hazardous mixtures). Unlike reversible household failures, these pose risks of catastrophic damage or injury in chemical settings.

Paired Contrastive Data. We generate paired success and failure trajectories under identical conditions (Fig. 3(d)). This rigorous alignment minimizes confounds, facilitating the learning of discriminative anomalous boundaries.

LabRobFail-Bench

To comprehensively evaluate robotic resilience, we introduce **LabRobFail-Bench**, the first multi-dimensional benchmark tailored for the high-stakes environment of chemical laboratories. As illustrated in Fig. 3(c), the benchmark assesses agents across **six evaluation dimensions** organized into **three progressive cognitive levels**: (1) *L1: Task Understanding*, (2) *L2: Failure Detection & Localization*, and (3) *L3: Failure Analysis & Correction*.

Problem Formulation. We define the failure analysis task as a mapping from multi-modal observations to a structured diagnostic report. Formally, given an input $\mathcal{X} = \langle \mathcal{V}, \mathcal{Q} \rangle$ (where $\mathcal{V}$ denotes the RGB video stream, $\mathcal{Q}$ is the task instruction), the model predicts a composite failure state $\mathcal{Y}$:

$$\mathcal{Y} = \underbrace{\langle y_{task} \rangle}_{L1}, \underbrace{\langle y_{dete}, y_{loca} \rangle}_{L2}, \underbrace{\langle y_{type}, y_{risk}, y_{corr} \rangle}_{L3} \quad (3)$$

encompassing task status y_{task} , detection flags y_{dete} , temporal localization y_{loca} , failure type y_{type} , risk severity y_{risk} , and actionable correction policies y_{corr} .

L1: Task Understanding (Q1). Decomposes videos into atomic action sequences (e.g., *pick*, *pour*) and their temporal dependencies, providing structured context for subsequent anomaly reasoning.

L2: Failure Detection & Localization (Q2-Q3). Evaluates anomaly perception. **Q2** performs binary detection, while **Q3** localizes the failure to a specific frame number. Precise localization captures subtle transitions, enabling timely intervention before irreversible hazards occur.

L3: Failure Analysis & Correction (Q4-Q6). Targets deeper reasoning. **Q4** assigns failures to four severity levels: *Minor*, *Recoverable*, *Critical*, and *Catastrophic*. **Q5** classifies the failure type, while **Q6** outputs fine-grained, executable corrections over trajectory, orientation (ΔR), and gripper states to support closed-loop recovery.

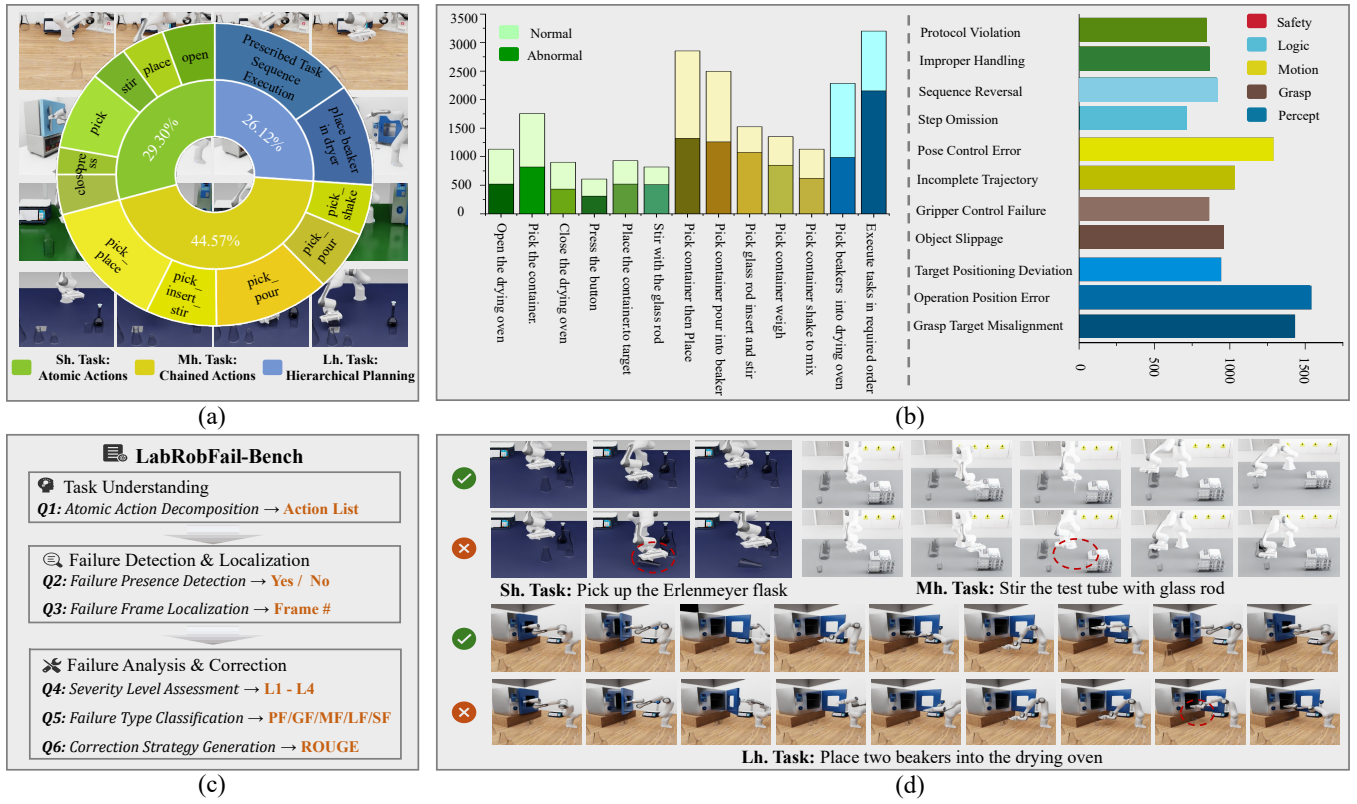


Figure 3: **LabRobFail-Data** and **LabRobFail-Bench**. (a) Distribution of short-, medium-, and long-horizon tasks, corresponding to atomic actions, chained actions, and hierarchical planning. (b) Distribution of normal and failed trajectories across tasks (left), and 11 fine-grained failure types grouped into five categories (right). (c) Six-dimensional benchmark covering task understanding, failure detection and localization, and failure analysis and correction. (d) Representative success-failure trajectory pairs at each task complexity level.

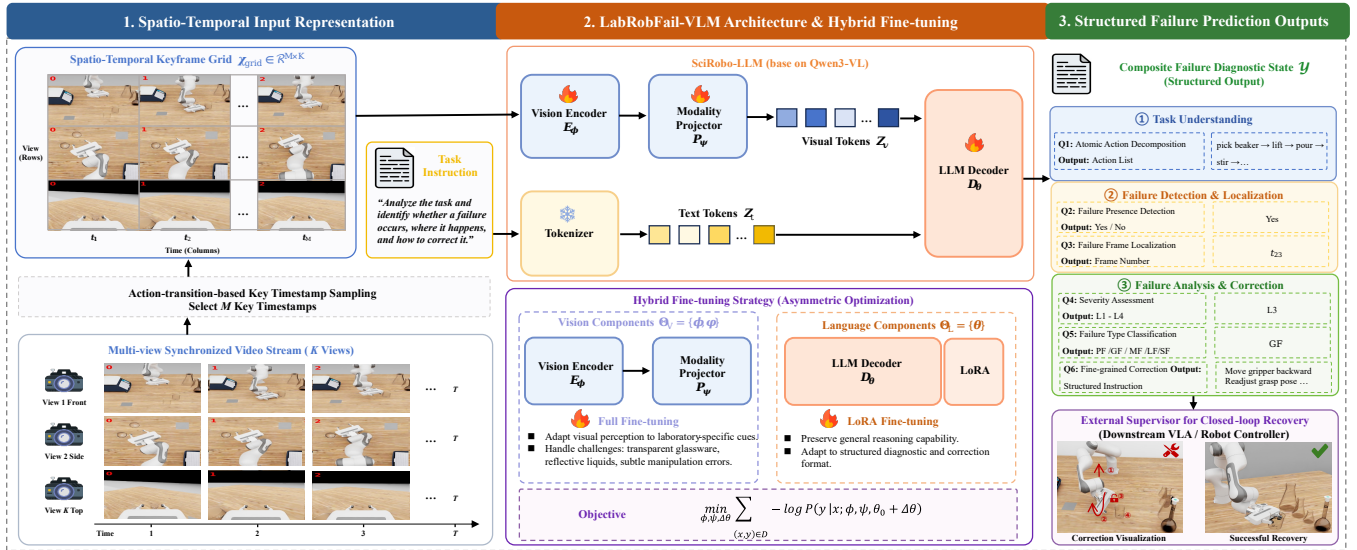


Figure 4: (a) LabRobFail-VLM Training: Qwen3-VL is fine-tuned for task understanding, failure detection, and correction reasoning. (b) VLA Correction & Recovery: fine-grained correction instructions enable VLA models to recover from failures.

LabRobFail-VLM

We develop **LabRobFail-VLM**, a specialized vision-language model for laboratory failure analysis, as shown in

Fig. 4. Rather than reducing failure recognition to binary classification, LabRobFail-VLM formulates it as a structured,

recovery-oriented prediction task that jointly supports task understanding, failure detection and localization, failure diagnosis, and corrective guidance.

Spatio-Temporal Input Representation

Standard VLMs struggle with long-horizon, multi-view robotic videos due to excessive visual tokens. We therefore represent each trajectory as a compact *Spatio-Temporal Keyframe Grid* that preserves both temporal progression and synchronized multi-view observations. Formally, let $\mathcal{V} = \{V^{(k)}\}_{k=1}^K$ denote synchronized video streams from K viewpoints, where $V^{(k)} = \{I_1^{(k)}, \dots, I_T^{(k)}\}$ is the frame sequence captured from the k -th view. We first sample M key timestamps $\{t_1, \dots, t_M\}$ according to action transitions, so that the selected frames summarize the critical stages of the manipulation process. We then construct an image grid $X_{\text{grid}} \in \mathbb{R}^{H \times W \times 3}$ by arranging the sampled frames into an $M \times K$ matrix, where rows correspond to temporal steps and columns correspond to viewpoints. We also render the temporal index t at the top-left of each image, allowing explicit time reference during failure localization across views.

Architecture and Hybrid Fine-tuning

As shown in Fig. 4, LabRobFail-VLM is built upon Qwen3-VL (Bai et al. 2025) and adapted to the laboratory failure-analysis setting through a hybrid fine-tuning strategy. The model consists of a vision encoder E_ϕ , a modality projector P_ψ , and a large language model decoder D_θ . Given an input tuple $\mathcal{X} = (X_{\text{grid}}, Q_{\text{instruct}})$, the model first extracts visual and textual representations as

$$Z_v = P_\psi(E_\phi(X_{\text{grid}})), \quad Z_t = \text{Tokenizer}(Q_{\text{instruct}}), \quad (4)$$

and then autoregressively predicts the failure output:

$$P(\mathcal{Y} | \mathcal{X}) = \prod_{j=1}^L D_\theta(y_j | y_{<j}, Z_v, Z_t), \quad (5)$$

where $\mathcal{Y}$ denotes the composite failure diagnostic state defined in Eq. (3).

Hybrid Fine-tuning Strategy. A key challenge is the domain gap between web-scale pretraining and laboratory observations, where transparent glassware, reflective liquids, and subtle manipulation errors are underrepresented. To improve domain adaptation while preserving language reasoning, we adopt an asymmetric optimization strategy.

Specifically, we divide the model parameters into vision-related and language-related components $\Theta_V = \{\phi, \psi\}$ and $\Theta_L = \{\theta\}$. We apply *full fine-tuning* to Θ_V so that the visual encoder and projector can better capture laboratory-specific perceptual cues. In contrast, for Θ_L , we employ LoRA (Hu et al. 2022) to adapt the language model to our structured diagnostic and correction format while preserving its general reasoning capability. The resulting training objective is

$$\min_{\phi, \psi, \Delta\theta} \sum_{(\mathcal{X}, \mathcal{Y}) \in \mathcal{D}} -\log P(\mathcal{Y} | \mathcal{X}; \phi, \psi, \theta_0 + \Delta\theta), \quad (6)$$

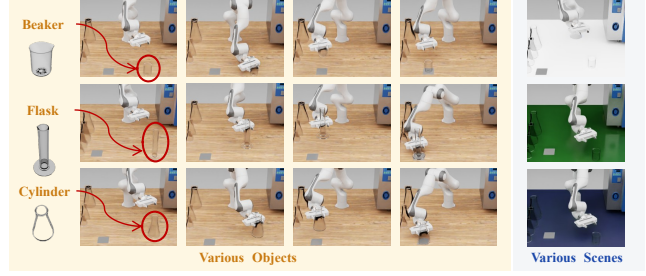


Figure 5: Examples of objects and scene used for evaluating generalization in the Unseen setting.

where θ_0 denotes the frozen pretrained model parameters, and $\Delta\theta$ is the low-rank adaptation update. This hybrid strategy enables LabRobFail-VLM to strengthen domain-specific visual perception while retaining stable high-level reasoning for structured failure diagnosis and correction.

Experiment

Experimental Setup

Implementation Details. To address the significant domain gap in laboratory scenarios, we employ a full fine-tuning strategy on Qwen3-VL-8B (Bai et al. 2025). We utilize differentiated learning rates (2×10^{-6} for the vision encoder, 1×10^{-5} for the projector and backbone) and train for 3 epochs with a global batch size of 256. The optimization uses a cosine scheduler (warmup 0.03) on 8 NVIDIA H100 GPUs via DeepSpeed ZeRO-3.

Evaluation Metrics. We report *Top-1 Accuracy* for tasks Q1–Q5 (task understanding, detection, localization, severity, and classification). For correction (Q6), we employ *BLEU-n* ($n = 1 \dots 4$) and *ROUGE-L* to quantify lexical alignment and semantic consistency with ground-truth references.

Quantitative Experimental Results

To comprehensively evaluate model performance and robustness, we partition the test set into two distinct subsets: **Seen** (overlapping with training domains) and **Unseen** (containing novel scenes and target objects). Fig. 5 shows examples of objects and scenes used in the Unseen evaluation setting.

Performance on Seen Environments. LabRobFail-VLM consistently outperforms all baselines across the six evaluation dimensions. It achieves **92.58%** accuracy in failure detection (Q2) and **85.58%** in temporal localization (Q3), while exceeding **90%** on severity assessment and failure classification (Q4–Q5). For correction generation (Q6), it reaches **0.74** BLEU-4 and **0.89** ROUGE-L, demonstrating the importance of domain-specific adaptation for reliable failure diagnosis and actionable correction.

Generalization to Unseen Environments. Table 3 reports generalization under three progressively harder settings: novel objects (*Object*), novel scenes (*Scene*), and both combined (*Both*). LabRobFail-VLM consistently outperforms generalist VLMs across all settings. In the most challenging *Both* setting, it still reaches **76.34%** on Temporal Localization (Q3), compared with 11.12% and 4.75% for Gemini-2.5-flash and Qwen3-VL-8B. It also maintains substantially

Table 2: Main results on LabRobFail-Bench (Seen Environments). Q1-Q5 denote Task Understanding, Failure Detection, Temporal Localization, Severity Assessment, and Failure Classification (accuracy %). Q6 denotes Correction Strategy (BLEU and ROUGE-L). Best results are in **bold**.

Method	Accuracy (%) $\uparrow$					Q6: Correction Strategy $\uparrow$				
	Q1	Q2	Q3	Q4	Q5	BLEU-1	BLEU-2	BLEU-3	BLEU-4	ROUGE-L
Qwen3-VL-8B (Bai et al. 2025)	47.45	59.50	5.59	68.82	35.88	0.2594	0.1249	0.0626	0.0366	0.2976
LLaVA-1.6-7B (Liu et al. 2024)	25.62	52.96	7.94	31.47	34.71	0.1923	0.0946	0.0463	0.0261	0.2567
LLaVA-1.6-13B (Liu et al. 2024)	35.80	52.96	13.24	33.53	31.18	0.2278	0.0999	0.0506	0.0255	0.2598
InternVL2.5-8B (Chen et al. 2024)	28.10	47.04	12.35	18.24	42.06	0.1174	0.0507	0.0258	0.0149	0.1430
DeepSeek-VL2-small (Wu et al. 2024)	34.25	47.51	13.82	58.82	33.18	0.1451	0.0852	0.0311	0.0182	0.1818
Gemini-2.0-flash (Team et al. 2024)	26.98	56.85	6.76	44.41	36.89	0.1161	0.0540	0.0282	0.0160	0.1440
Gemini-2.5-flash (Comanici et al. 2025)	38.92	57.13	15.53	48.84	39.68	0.1174	0.0507	0.0258	0.0149	0.1430
GPT-5.4 (OpenAI 2026)	45.16	52.34	15.88	47.06	38.54	0.1194	0.0606	0.0307	0.0161	0.1520
LabRobFail-VLM (Ours)	94.45	90.83	77.21	83.18	73.21	0.7629	0.7569	0.7452	0.7362	0.7542

Table 3: Generalization results on unseen environments (novel objects (Object), novel scene backgrounds (Scene), and both (Both)).

Method	Setting	Accuracy (%) $\uparrow$					Q6: Correction Strategy $\uparrow$				
		Q1	Q2	Q3	Q4	Q5	BLEU-1	BLEU-2	BLEU-3	BLEU-4	ROUGE-L
Qwen3-VL-8B (Bai et al. 2025)	Object	58.24	64.86	5.29	57.18	26.20	0.2594	0.0885	0.0381	0.0246	0.2731
	Scene	61.75	68.27	4.44	59.50	45.65	0.2474	0.1279	0.0707	0.0384	0.2748
	Both	56.09	62.83	4.75	43.28	40.66	0.2196	0.0717	0.0298	0.0201	0.2426
LLaVA-1.6-13B (Liu et al. 2024)	Object	38.28	53.12	15.11	34.18	33.75	0.2331	0.0974	0.0527	0.0266	0.2459
	Scene	52.54	61.45	15.81	38.37	29.13	0.2119	0.0901	0.0459	0.0234	0.2317
	Both	39.12	55.24	14.21	36.11	30.14	0.2108	0.0852	0.0512	0.0257	0.2427
Gemini-2.5-flash (Team et al. 2024)	Object	38.12	56.27	12.53	48.84	34.34	0.1214	0.0524	0.0425	0.0241	0.1524
	Scene	39.24	59.21	14.24	50.24	38.98	0.1304	0.0545	0.0398	0.0197	0.1672
	Both	39.12	56.54	11.12	48.54	39.12	0.1245	0.0721	0.0456	0.0241	0.1587
LabRobFail-VLM (Ours)	Object	91.53	79.15	60.22	53.05	59.86	0.4876	0.4536	0.3974	0.3492	0.4668
	Scene	93.42	85.56	74.39	63.51	72.63	0.6793	0.6276	0.5805	0.5929	0.6604
	Both	89.09	71.02	41.41	46.02	38.72	0.3981	0.3575	0.3735	0.3269	0.3894

Table 4: Success rate of downstream VLA recovery tasks.

Task	Baseline	Rate(%)	+Ours	Rate(%)	Improv.
Pick	OpenVLA	76	19/25	84	+8%
Place	OpenVLA	48	13/25	52	+4%
Pour	OpenVLA	16	7/25	28	+12%
Open drying oven	OpenVLA	28	10/25	40	+12%
Close drying oven	OpenVLA	12	5/25	20	+8%
Pick	ACT	64	17/25	68	+4%
Place	ACT	52	15/25	60	+8%
Pour	ACT	32	12/25	48	+16%

Table 5: Effect of pretraining on LabRobFail-Data for AHA model.

Dataset	w/o Aux	w/ Aux
AHA	59.87	68.85
AHA + LabRobFail-Data (Ours)	63.96	72.74

better correction quality on Q6 (BLEU-4 0.3459 vs. ~ 0.02), suggesting that the model captures transferable failure dynamics rather than merely fitting scene-specific visual cues.

Downstream VLA Recovery

To evaluate closed-loop recovery, we integrate LabRobFail-VLM as an **external supervisor** of OpenVLA (Kim et al. 2024), as shown in Fig. 4(b). The model monitors execution in real time and generates fine-grained corrections for arm trajectories, gripper states, and execution conditions. A deterministic **Action Dictionary** maps these semantic instructions to executable control primitives without additional

fine-tuning. As shown in Table 4, this framework improves *Place* success rates by **8 percentage points** and doubles *Pour* performance from **32% to 48%**, demonstrating more reliable recovery from laboratory execution failures.

Transferability Value of LabRobFail-Data

To validate the transferability of LabRobFail-Data, we pre-train the AHA model (Duan et al. 2024) using LabRobFail-Data. Table 5 shows consistent improvements: accuracy rises by **+4.1%** (59.87% $\rightarrow$ 63.96%) in isolation and by **+3.9%** (68.85% $\rightarrow$ 72.74%) even when combined with large-scale auxiliary datasets. This shows that LabRobFail-Data provides unique, complementary failure semantics (e.g., such as liquid dynamics) that act as critical prior knowledge to enhance generalizable robustness.

Conclusion and Limitations

Conclusion. We introduce LabRobFail, a comprehensive framework comprising a physics-based simulation platform (LabRobFail-Sim), a large-scale dataset of 20K+ trajectories (LabRobFail-Data), and a multi-dimensional benchmark (LabRobFail-Bench). Our specialized model, LabRobFail-VLM, outperforms generalist baselines in failure reasoning and facilitates robust closed-loop recovery in downstream VLA tasks, paving the way for reliable autonomous discovery.

Limitations. While LabRobFail demonstrates robust capabilities in real-time failure detection and fine-grained cor-

rection, several limitations remain. First, although we validated sim-to-real transfer on physical robots, our training data relies entirely on synthetic environments. Bridging the visual fidelity gap remains a key challenge for large-scale real-world deployment. Second, the current correction mechanism employs a deterministic action dictionary to map semantic instructions to robot actions, which limits the flexibility of recovery behaviors. Future work aims to scale up VLA experiments and investigate end-to-end policy adaptation, enabling robots to directly interpret and execute open-ended natural language corrections.

References

- Abolhasani, M.; and Kumacheva, E. 2023. The rise of self-driving labs in chemical and materials sciences. *Nature Synthesis*, 2(6): 483–492.
- Bai, S.; Cai, Y.; Chen, R.; Chen, K.; Chen, X.; Cheng, Z.; Deng, L.; Ding, W.; Gao, C.; Ge, C.; et al. 2025. Qwen3-VL Technical Report. *arXiv preprint arXiv:2511.21631*.
- Black, K.; Brown, N.; Darpinian, J.; Dhabalia, K.; Driess, D.; Esmail, A.; Equi, M.; Finn, C.; Fusai, N.; et al. 2025. $\pi_{0.5}$: A Vision-Language-Action Model with Open-World Generalization. *arXiv preprint arXiv:2504.16054*.
- Chen, Z.; Wu, J.; Wang, W.; Su, W.; Chen, G.; Xing, S.; Zhong, M.; Zhang, Q.; Zhu, X.; Lu, L.; et al. 2024. Internvl: Scaling up vision foundation models and aligning for generic visual-linguistic tasks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 24185–24198.
- Comanici, G.; Bieber, E.; Schaekermann, M.; Pasupat, I.; Sachdeva, N.; Dhillon, I.; Blistein, M.; Ram, O.; Zhang, D.; Rosen, E.; Marris, L.; Petulla, S.; Gaffney, C.; Aharoni, A.; Lintz, N.; Pais, T. C.; Jacobsson, H.; Szepektor, I.; Jiang, N.-J.; Haridasan, K.; Omran, A.; et al. 2025. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*.
- Duan, J.; Pumacay, W.; Kumar, N.; Wang, Y. R.; Tian, S.; Yuan, W.; Krishna, R.; Fox, D.; Mandlekar, A.; and Guo, Y. 2024. Aha: A vision-language-model for detecting and reasoning over failures in robotic manipulation. *arXiv preprint arXiv:2410.00371*.
- Epps, R. W.; Bowen, M. S.; Volk, A. A.; Abdel-Latif, K.; Han, S.; Reyes, K. G.; Amassian, A.; and Abolhasani, M. 2020. Artificial chemist: an autonomous quantum dot synthesis bot. *Advanced Materials*, 32(30): 2001626.
- Gislain, C.; Rahimi, H.; Sigaud, O.; and Chetouani, M. 2025. I-FailSense: Towards General Robotic Failure Detection with Vision-Language Models. *arXiv preprint arXiv:2509.16072*.
- Guo, Y.; Wang, Y.-J.; Zha, L.; and Chen, J. 2024. Doremi: Grounding language model by detecting and recovering from plan-execution misalignment. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 12124–12131. IEEE.
- Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; Chen, W.; et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2): 3.
- James, S.; Ma, Z.; Arrojo, D. R.; and Davison, A. J. 2020. Rlbench: The robot learning benchmark & learning environment. *IEEE Robotics and Automation Letters*, 5(2): 3019–3026.
- Kambhampati, S.; Valmeekam, K.; Guan, L.; Verma, M.; Stechly, K.; Bhambri, S.; Saldyt, L. P.; and Murthy, A. B. 2024. Position: LLMs can’t plan, but can help planning in LLM-modulo frameworks. In *Forty-first International Conference on Machine Learning*.
- Kim, M. J.; Pertsch, K.; Karamcheti, S.; Xiao, T.; Balakrishna, A.; Nair, S.; Rafailov, R.; Foster, E.; Lam, G.; Sanketi, P.; et al. 2024. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*.
- Lan, Z.; Jiang, Y.; Wang, R.; Xie, X.; Zhang, R.; Zhu, Y.; Li, P.; Yang, T.; Chen, T.; Gao, H.; et al. 2025. Autobio: A simulation and benchmark for robotic automation in digital biology laboratory. *arXiv preprint arXiv:2505.14030*.
- Li, R.; Hu, Z.; Qu, W.; Zhang, J.; Yin, Z.; Zhang, S.; Huang, X.; Wang, H.; Wang, T.; Pang, J.; et al. 2025. Lab-Utopia: High-Fidelity Simulation and Hierarchical Benchmark for Scientific Embodied Agents. *arXiv preprint arXiv:2505.22634*.
- Lin, J.; Du, Y.; Watkins, O.; Hafner, D.; Abbeel, P.; Klein, D.; and Dragan, A. 2023. Learning to model the world with language. *arXiv preprint arXiv:2308.01399*.
- Liu, H.; Li, C.; Li, Y.; Li, B.; Zhang, Y.; Shen, S.; and Lee, Y. J. 2024. Llvnext: Improved reasoning, ocr, and world knowledge.
- Liu, Z.; Bahety, A.; and Song, S. 2023. Reflect: Summarizing robot experiences for failure explanation and correction. *arXiv preprint arXiv:2306.15724*.
- Lu, W.; Ye, M.; Ye, Z.; Tao, R.; Yang, S.; and Zhao, B. 2025. RoboFAC: A Comprehensive Framework for Robotic Failure Analysis and Correction. *arXiv preprint arXiv:2505.12224*.
- Mitash, C.; Wang, F.; Lu, S.; Terhuja, V.; Garaas, T.; Polido, F.; and Nambi, M. 2023. Armbench: An object-centric benchmark dataset for robotic manipulation. *arXiv preprint arXiv:2303.16382*.
- OpenAI. 2026. GPT-5.4 Thinking system card. Technical report, OpenAI. <https://openai.com/index/gpt-5-4-thinking-system-card/>.
- Pacaud, P.; Garcia, R.; Chen, S.; and Schmid, C. 2025. Guardian: Detecting Robotic Planning and Execution Errors with Vision-Language Models. *arXiv preprint arXiv:2512.01946*.
- Qu, Y.; Zhang, T.; Garg, N.; and Kumar, A. 2024. Recursive introspection: Teaching LLM agents how to self-improve. In *ICML 2024 Workshop on Structured Probabilistic Inference* {\&} *Generative Modeling*.
- Steiner, S.; Wolf, J.; Glatzel, S.; Andreou, A.; Granda, J. M.; Keenan, G.; Hinkley, T.; Aragon-Camarasa, G.; Kitson, P. J.; Angelone, D.; et al. 2019. Organic synthesis in a modular robotic system driven by a chemical programming language. *Science*, 363(6423): eaav2211.
- Szymanski, N. J.; Rendy, B.; Fei, Y.; Kumar, R. E.; He, T.; Milsted, D.; McDermott, M. J.; Gallant, M.; Cubuk, E. D.;

Merchant, A.; et al. 2023. An autonomous laboratory for the accelerated synthesis of novel materials. *Nature*, 624(7990): 86–91.

Team, G.; Georgiev, P.; Lei, V. I.; Burnell, R.; Bai, L.; Gulati, A.; Tanzer, G.; Vincent, D.; Pan, Z.; Wang, S.; et al. 2024. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*.

Thoduka, S.; Hochgeschwender, N.; Gall, J.; and Plöger, P. G. 2024. A multimodal handover failure detection dataset and baselines. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 17013–17019. IEEE.

Walke, H. R.; Black, K.; Zhao, T. Z.; Vuong, Q.; Zheng, C.; Hansen-Estruch, P.; He, A. W.; Myers, V.; Kim, M. J.; Du, M.; et al. 2023. Bridgedata v2: A dataset for robot learning at scale. In *Conference on Robot Learning*, 1723–1736. PMLR.

Wu, Z.; Chen, X.; Pan, Z.; Liu, X.; Liu, W.; Dai, D.; Gao, H.; Ma, Y.; Wu, C.; Wang, B.; et al. 2024. Deepseek-vl2: Mixture-of-experts vision-language models for advanced multimodal understanding. *arXiv preprint arXiv:2412.10302*.

Zeng, X.; Zhou, X.; Li, Y.; Shi, J.; Li, T.; Chen, L.; Ren, L.; and Li, Y.-L. 2025. Diagnose, Correct, and Learn from Manipulation Failures via Visual Symbols. *arXiv preprint arXiv:2512.02787*.

Zhen, H.; Qiu, X.; Chen, P.; Yang, J.; Yan, X.; Du, Y.; Hong, Y.; and Gan, C. 2024. 3d-vla: A 3d vision-language-action generative world model. *arXiv preprint arXiv:2403.09631*.

Zhou, E.; Su, Q.; Chi, C.; Zhang, Z.; Wang, Z.; Huang, T.; Sheng, L.; and Wang, H. 2025. Code-as-monitor: Constraint-aware visual programming for reactive and proactive robotic failure detection. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, 6919–6929.

Zitkovich, B.; Yu, T.; Xu, S.; Xu, P.; Xiao, T.; Xia, F.; Wu, J.; Wohlhart, P.; Welker, S.; Wahid, A.; et al. 2023. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, 2165–2183. PMLR.

SciRobo-Sim

LLM Annotation Pipeline

VQA Generation Module To ensure the large language model accurately understands our requirements, we first assign it a specific role through a system prompt, establishing its expertise in laboratory robotics annotation:

“You are an expert annotator for robotic manipulation tasks in chemical laboratory environments. For each dimension, randomly select ONE question variant from the provided options, then generate the corresponding answer based on the visual observation and metadata.”

Subsequently, we provide a standardized image description template to help the model understand the spatio-temporal structure of the input keyframe grid:

“The image consists of a series of sequential frames. Each column presents a different camera viewpoint, and each row corresponds to a later time step, with the time-step label shown in the top-left corner. These frames depict the robotic arm’s motion during Task: {task_name}. For each of the following six dimensions, select ONE question from the provided variants and answer it.”

Regarding question design, we provide multiple question variants for each evaluation dimension, as shown in Table 6, to ensure diversity in the generated QA pairs. During annotation, GPT-4o randomly selects one variant per dimension, then generates the corresponding answer according to our provided configuration file.

For the answer options, we define fixed categories for Q4 and Q6 to ensure annotation consistency, as shown in Table 7. The model reads these predefined options from our configuration file and selects the most appropriate category based on the observed failure context.

Based on the configuration file, GPT-4o generates complete question-answer pairs for each trajectory. We then use a post-processing script to convert the raw outputs into our standardized annotation format. The following shows an example of the final formatted QA pair:

```

1  {
2    "id": "10_Q4",
3    "image": "close_drying_oven_episode_0_erro.png",
4    "conversations": [
5      {
6        "from": "human",
7        "value": "<image>\n\nThe image consists of a
          series of sequential frames...What is the
          severity level of this error?\nOptions:\n
          (1) Risk Level\n(2) Dangerous Level\n(3)
          Fatal Level\n(4) Management Incident"
8      },
9      {"from": "gpt", "value": "2"}
10   ]
11 }

```

```

1  {
2    "id": "10_Q6",
3    "image": "close_drying_oven_episode_0_erro.png",
4    "conversations": [

```

```

5  {
6    "from": "human",
7    "value": "<image>\n\nThe image consists of a
          series of sequential frames...What is the
          correction solution for this failure?"
8  },
9  {"from": "gpt", "value": "Grasp the door handle
          with the gripper, move the robotic arm
          forward to close the door."}
10 ]
11 }

```

The remaining dimensions (Q1-Q3, Q5) follow the same format, with each question-answer pair stored as an independent training sample.

Correction Strategy Formulation We design a set of fine-grained correction instructions that can be mapped to executable robot actions. Each instruction follows a unified template structure containing four components: position adjustment, orientation adjustment, gripper state, and execution condition. Table 8 presents the complete set of correction strategies organized by operation type.

SciRobo-Data

SciRobo-Data encompasses a comprehensive failure taxonomy covering 5 major categories (Safety, Logic, Motion, Grasp, and Percept) and 11 fine-grained failure types. To provide a clear understanding of each failure type, we present a representative example for each failure category below, along with the corresponding visualization results. Each example shows the keyframe sequence where the failure occurs, with the failure frame highlighted. The 11 failure types include: (1) *Protocol Violation* and *Improper Handling* under Safety; (2) *Sequence Reversal* and *Step Omission* under Logic; (3) *Pose Control Error* and *Incomplete Trajectory* under Motion; (4) *Gripper Control Failure* and *Object Slippage* under Grasp; and (5) *Target Positioning Deviation*, *Operation Position Error*, and *Grasp Target Misalignment* under Percept.

Safety Failure

Protocol Violation. The stirring task consists of the following sequence: first, the glass rod is grasped from the test tube rack, then transported to a position above the beaker, inserted into the beaker, and subsequently oscillated gently in a lateral manner. A Protocol Violation occurs during the insertion phase, where an angular misalignment prevents the glass rod from being correctly inserted into the beaker, resulting in insufficient stirring that violates laboratory operation protocols.



Figure 6: Visualization of Protocol Violation failure.

Improper Handling. The pouring operation from the Erlenmeyer flask into the beaker consists of four sequential

Table 6: Question variants for each evaluation dimension. GPT-4o randomly selects one variant per dimension during annotation.

Dimension	Question Variants
Q1: Task Understanding	- "What atomic actions does the current task consist of? Please choose from: grasp, place, open, close, pour, shake, insert, stir, and list them in execution order." - "Break down the robot's operation into primitive actions. Available primitives: grasp, place, open, close, pour, shake, insert, stir." - "Identify the sequence of manipulation primitives performed in this task. Options: grasp, place, open, close, pour, shake, insert, stir." - "Which atomic actions are executed in this task and in what order? Choose from: grasp, place, open, close, pour, shake, insert, stir."
Q2: Failure Detection	- "Is there any error in this task? (Yes/No)" - "Does the robot successfully complete the intended operation? (Yes/No)" - "Did anything go wrong during the execution? (Yes/No)" - "Is there any anomaly or failure observed in this trajectory? (Yes/No)" - "Was this task executed without issues? (Yes/No)"
Q3: Temporal Localization	- "At which frame does the failure occur? (Refer to the frame number in the upper left corner)" - "Looking at the frame indices, when exactly does the error happen?" - "Identify the timestep where the failure first manifests. (Use the frame number displayed)" - "At what point in the sequence does the anomaly appear? (Specify the frame number)" - "Which frame marks the onset of the failure?"
Q4: Severity Assessment	- "What is the severity level of this error?" - "How severe is this failure in terms of laboratory safety?" - "Assess the risk level of this failure." - "Rate the severity of this anomaly."
Q5: Failure Classification	- "What is the failure type? Select the most appropriate category." - "How would you classify the root cause of this failure?" - "Which category best describes this anomaly?" - "Identify the failure mode from the following categories."
Q6: Correction Strategy	- "What is the correction solution for this failure?" - "How should the robot recover from this situation?" - "Propose a specific corrective action to address this failure." - "What adjustments should be made to fix this error?" - "Describe the recovery procedure for this anomaly."

Table 7: Answer options for Q4 (Severity Assessment) and Q5 (Failure Classification).

Dim.	Option	Definition
Q4	(1) Risk Level	Collision, liquid spillage, or potential falling
	(2) Dangerous Level	Requires manual intervention to proceed
	(3) Fatal Level	Equipment damage, requiring manual intervention
	(4) Management Incident	Non-compliant operations (e.g., unclosed cabinet)
Q5	(1) Perception Failure	Failed to locate target objects or positions
	(2) Grasping Failure	Object slippage or unstable grasp
	(3) Motion Failure	Motion execution incomplete
	(4) Logic Failure	Incorrect order or omitted steps
	(5) Safety Failure	Violating laboratory safety protocols

phases: grasping the flask, transporting it to a position above the beaker, executing the pouring motion, and subsequently returning the flask to its original location. An Improper Handling event occurs during the returning phase, wherein the flask fails to be placed back at its designated original position, violating the chemical laboratory requirement for proper equipment placement after use, thereby leading to an abnormal task outcome.



Figure 7: Visualization of Improper Handling failure.

Logic Failure

Sequence Reversal. The stirring task consists of the following sequence: first, the robotic arm grasps the glass rod from

the test tube rack, then moves it to a position above the beaker, inserts the rod into the beaker, and performs a lateral stirring motion. A Sequence Reversal event occurs at the beginning of the task, where the robotic arm executes a stirring motion without the glass rod prior to grasping it, leading to an incorrect task execution.



Figure 8: Visualization of Sequence Reversal failure.

Step Omission. The stirring task consists of the following sequence: first, the glass rod is grasped from the test tube rack, then transported to a position above the beaker, inserted into the beaker, and subsequently oscillated gently in a lateral manner. Step Omission occurs during the initial phase of the task when the robotic arm proceeded directly to stirring without gripping the glass rod.

Table 8: Fine-grained correction strategies organized by failure category. Placeholders in {brackets} are filled based on specific task context.

Category	Correction Instruction
Safety Failure	Keep the robotic arm position unchanged, maintain gripper state, and keep the arm horizontal to complete the experiment safely. After completing the operation, keep the gripper horizontal, move the robotic arm, and place the {target} steadily.
Logic Failure	First move the robotic arm to grasp the {target} with gripper closed, then move to the {destination} and open the gripper to place. Pause current execution, return to perform the omitted {action}, then resume the remaining workflow.
Motion Failure	Adjust the gripper orientation to horizontal, keep the gripper closed, then move the robotic arm forward to insert into the {target}. Keep the gripper closed and orientation unchanged, move the robotic arm side to side repeatedly until the liquid is fully stirred. Keep the gripper closed on the door handle and move the robotic arm {direction} until the door is fully {open/closed}. Move the robotic arm to align the {source} above the {destination}, keep the gripper closed, then progressively tilt the gripper until the liquid is fully poured.
Grasp Failure	Keep the gripper open and move the robotic arm until the {target} is centered within the gripper, then close the gripper to secure the grasp. Move the robotic arm toward the {target}, adjust the gripper orientation to horizontal, keep the gripper open until centered, then close the gripper to grasp. Keep the gripper closed to secure the {target} and move the robotic arm to the {destination} while maintaining orientation horizontal.
Percept Failure	Move the robotic arm to center the gripper on the door handle, close the gripper to grasp, then move {direction} to {open/close} the door. Move the robotic arm to position the {target} above the {destination}, keep the gripper orientation horizontal, then open the gripper to release. Move the robotic arm to align the gripper with the {target}, keep the gripper closed, then move forward until pressed. Open the gripper to release, reposition the robotic arm to center the {target}, then close the gripper firmly to secure.
Manual Operation	The object has fallen or critical failure occurred, please correct it manually right away.



Figure 9: Visualization of Step Omission failure.

Motion Failure

Pose Control Error. The placement task involves moving the beaker to a position above the target location. A Pose Control Error occurs during the beaker placement phase, where an abnormal twist in the robot arm’s pose leads to unsuccessful placement of the beaker at the target location, resulting in task failure.



Figure 10: Visualization of Pose Control Error failure.

Incomplete Trajectory. The button-pressing task requires the robotic arm to move to the designated red button and

execute the press. An Incomplete Trajectory occurs during the approach phase: the arm stops prematurely and remains stationary in front of the button, causing task failure.



Figure 11: Visualization of Incomplete Trajectory failure.

Grasp Failure

Gripper Control Failure. The sequence begins with grasping the chemical instrument. A Gripper Control Failure manifests during the grasping phase. Inadequate gripper closure aperture prevents successful object acquisition, leading to task failure.



Figure 12: Visualization of Gripper Control Failure.

Object Slippage. The placement task involves moving the chemical equipment to a designated location. The sequence consists of grasping the beaker, moving it above the target, and placing it at the target. An Object Slippage event occurs during the transport phase, where insufficient gripper force causes the beaker to slip and fall, resulting in task failure.

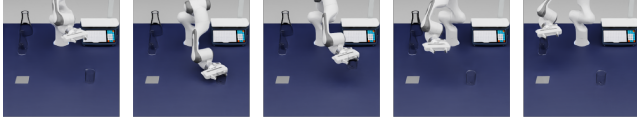


Figure 13: Visualization of Object Slippage failure.

Percept Failure

Target Positioning Deviation. The placement task involves the robotic arm first grasping the beaker and then placing it at the target location. A Target Positioning Deviation occurs during the final phase of the task, where the beaker is not placed within the designated target region, resulting in an abnormal outcome.



Figure 14: Visualization of Target Positioning Deviation failure.

Operation Position Error. The door-opening task involves moving the robotic arm to the front of the door handle, grasping the handle, and then pulling it backward in a fan-shaped motion. An Operation Position Error occurs when the gripper moves to the front of the door handle, where misalignment between the gripper and the handle position causes the door-opening operation to fail.



Figure 15: Visualization of Operation Position Error failure.

Grasp Target Misalignment. The sequence begins with grasping the chemical instrument. A Grasp Target Misalignment manifests during the grasping phase. Inadequate gripper closure aperture prevents successful object acquisition, leading to task failure.



Figure 16: Visualization of Grasp Target Misalignment failure.

Experiment Details

Training Details

We adopt Qwen3-VL-8B as the backbone and apply a hybrid fine-tuning strategy: full fine-tuning for the vision encoder and modality projector to adapt to laboratory-specific visual features (e.g., transparent glassware, reflective liquid surfaces), and LoRA for the LLM decoder to preserve general reasoning capabilities. The LoRA configuration uses rank $r = 64$ and scaling factor $\alpha = 128$. The input Spatio-Temporal Keyframe Grid has a resolution of 1536×768 pixels. We split SciRobo-Data into training, validation, and test sets with a ratio of 8:1:1. For the Seen subset, we employ random splitting to divide the dataset. For the Unseen subset, we perform diversity expansion on target objects and scene backgrounds to ensure inconsistency with the training distribution. Other training configurations (learning rates, batch size, optimizer) follow Section 5.1 of the main paper.

Action Dictionary

To bridge the gap between SciRobo-VLM’s semantic correction instructions and executable robot control, we construct an Action Dictionary that maps natural language descriptions to low-level control primitives for the 8-DoF Franka robot. Table 9 and Table 10 present the complete mapping organized by action type.

It is worth noting that while some of our semantic outputs cannot be directly mapped to low-level robot actions (e.g., high-level descriptions involving target objects or conditional execution), they provide rich contextual and positional information that enables downstream Vision-Language-Action (VLA) models to interpret and execute the intended corrections. This design choice lays the foundation for future work on end-to-end policy adaptation, where VLA models can directly consume natural language correction instructions without requiring a deterministic action dictionary.

Table 9: Action Dictionary: Position and Orientation mappings.

Semantic Instruction	Executable Action
<i>Position Actions</i>	
keep the robotic arm position unchanged	hold_position()
move the robotic arm forward	delta_pos(x=+d, y=0, z=0)
move the robotic arm backward	delta_pos(x=-d, y=0, z=0)
move the robotic arm {direction}	delta_pos(direction, dist)
move the robotic arm side to side repeatedly	oscillate(axis='y', amp, n)
move the robotic arm to center the gripper on {target}	align_to(target_id)
move the robotic arm to position {target}	move_above(dest_id, h)
above {destination}	
move the robotic arm to align {source}	align_above(src, dest)
above {destination}	
return to perform the omitted {action}	move_to_pose(pose_id)
reposition the robotic arm to center the {target}	align_to(target_id)
<i>Orientation Actions</i>	
keep the arm horizontal	set_orient(roll=0, pitch=0)
maintain orientation unchanged	hold_orientation()
adjust the gripper orientation to horizontal	set_orient(roll=0, pitch=0)
tilt the gripper	rotate(axis='y', angle= θ)
progressively tilt until liquid is fully poured	tilt_pour(angle= θ , v)

Table 10: Action Dictionary: Gripper action mappings.

Semantic Instruction	Executable Action
maintain gripper state	hold_gripper()
open the gripper	gripper_open()
close the gripper	gripper_close()
close the gripper to secure the grasp	gripper_close()
close the gripper firmly to secure	gripper_close()
open the gripper to release	gripper_open()